

TP M2-3 — Bluetooth basse consommation avec votre smartphone (nRF Connect)

Module 2 — vendredi 9 octobre 2026 — 15:30-16:15 (45 min), en équipe MBA Santé, IA & IoT
— Epitech — Nicolas LAURIO

En bref — ce que nous allons faire

- **Objectif** : comprendre comment un capteur cardiaque envoie la fréquence cardiaque à un smartphone par Bluetooth. Puis savoir lire ce message, case par case.
- **Ce que vous allez faire** :
 1. Repérer les appareils Bluetooth autour de vous (Étape 1) → un tableau de 5 appareils, et ce que leur signal révèle.
 2. Comprendre comment le capteur range ses données (Étape 2) → l'image « rayons et articles » en tête.
 3. Recevoir une fréquence cardiaque sur un téléphone, par l'une des 3 voies (Étape 3) → une capture d'écran nRF Connect.
 4. Décoder 3 messages du capteur à la main, avec un modèle à compléter (Étape 4, exercice central) → un tableau rempli : FC, contact, énergie, intervalles RR.
 5. Vérifier vos réponses avec Python (Étape 5) → le message « Tous les tests sont passés. » et le fichier `trames_decodees.csv`.
 6. Faire le lien avec le projet 1 (Étape 6, lecture) → savoir ce que votre ESP32 devra envoyer.
- **À la fin, vous déposez dans** `analyses/m2_tp3/` : `trames_decodees.csv` + votre tableau des 3 trames fait à la main, `capture_nrfconnect.png` (noms floutés) et `README.md` (voie choisie, tableau des 5 appareils, réponses aux questions).
- **Durée** : 45 min (15:30-16:15), en équipe.
- **Outils** : un smartphone avec l'application gratuite nRF Connect, le notebook `TP_M2_3_BLE_decodeur.ipynb` dans Google Colab. Aucun matériel du kit.
- **Si vous êtes bloqués** : à l'étape 3, prenez la voie C (trames fournies, faisable par tous) ; sinon, message dans le canal écrit de la promo avec une capture d'écran.

Trames **fictives** ou issues de **vos propres** appareils, usage **pédagogique**, aucune finalité

diagnostique. Vous n'utilisez **aucun matériel** du kit (livré le 30/10) : un smartphone suffit.

Organisation à distance. Le TP se fait en visio : l'intervenant n'est pas dans la salle.

- Questions : **canal écrit de la promo**, avec une capture d'écran (nRF Connect ou erreur Python).
- Le Bluetooth basse consommation porte à quelques mètres. Tout se passe donc **entre les téléphones de votre équipe**, sur votre site. Rien ne passe par Internet.

Version étudiants. Pour copier du code, partez du notebook ou du fichier Markdown fournis avec ce PDF : le copier-coller depuis un PDF peut altérer l'indentation ou couper des lignes.

1. Objectifs

À la fin du TP, vous saurez :

1. repérer les appareils Bluetooth basse consommation (BLE) autour de vous, et lire ce qu'ils diffusent : nom, force du signal (RSSI), données d'annonce ;
2. dire ce qu'une annonce Bluetooth peut révéler sur une personne (**RGPD**) ;
3. expliquer comment un capteur range ses données (**GATT**) : des rayons (services) qui contiennent des articles (caractéristiques). Exemple : le service normalisé **Heart Rate** (fréquence cardiaque, 0x180D) ;
4. décoder **à la main** un message du capteur (**Heart Rate Measurement**, 0x2A37), case par case, puis vérifier avec Python ;
5. faire le lien avec le projet 1 : votre ESP32 enverra lui-même ce message.

2. Prérequis

- Apport « La connectivité en bref » (14:45-15:15) : annonce, scan, connexion, services et caractéristiques.
- Pas besoin d'être matheux. L'étape 4 montre pas à pas comment lire un nombre écrit en hexadécimal ($0x48 = 72$) ou en interrupteurs ($0001\ 0110$).
- Python : Google Colab (recommandé) ou installation locale (détails : voir l'annexe).

3. Outils

Outil	À quoi il sert	Lien
nRF Connect for Mobile (Nordic Semiconductor, gratuit)	Voir les appareils Bluetooth, s'y connecter, lire leurs données ; sous Android, le téléphone peut aussi jouer le rôle d'un capteur	Android : https://play.google.com/store/apps/details?id=no.nordicsemi.android.mcp — iOS : https://apps.apple.com/app/id1054362403
Notebook <code>TP_M2_3_BLE_decodeur.ipynb</code>	Vérifier votre décodage (outil <code>decode_hrm</code>)	Teams / Moodle
(Option iOS) LightBlue (Punch Through, gratuit)	Faire jouer à un iPhone le rôle d'un capteur (« Virtual Devices »)	https://apps.apple.com/app/id557428110

Ouvrir le notebook dans Google Colab

1. Téléchargez `TP_M2_3_BLE_decodeur.ipynb` depuis Teams / Moodle.
2. Allez sur <https://colab.research.google.com> → *Fichier* → *Importer un notebook*.
3. Lancez *Exécution* → *Tout exécuter*.

Le notebook ne contient et n'envoie aucune donnée personnelle. Installation locale de Python : voir l'annexe.

Permissions du smartphone : autorisez le Bluetooth pour nRF Connect. Sous Android, autorisez aussi « Appareils à proximité » (Android 12 et plus) ou la **localisation** (versions plus anciennes). Sinon, la liste reste vide.

4. Règles du jeu : sécurité et RGPD

- Appareils qui ne sont pas à vous : vous **regardez** seulement ce qu'ils diffusent. Vous ne vous **connectez pas**.
- Connexion autorisée seulement aux appareils **de votre équipe**, avec l'accord de leur propriétaire.
- N'écrivez **jamais** dans une caractéristique d'un appareil : vous risquez de le dérégler ou de le bloquer. N'utilisez **pas** la fonction **DFU** de nRF Connect (mise à jour du logiciel interne de l'appareil).
- Sur les captures d'écran déposées dans le repo, **floutez** tout nom qui identifie une personne (« Montre de Prénom », etc.).

- Une fréquence cardiaque liée à une personne est une **donnée de santé** (RGPD, art. 9). Déposez seulement les **trames** (sans nom), et seulement avec l'accord du porteur.

5. Étapes

Étape 1 — Scanner les appareils autour de vous (10 min)

Pourquoi : voir ce que les objets Bluetooth crient en permanence à tout le monde, et ce que cela révèle sur leur propriétaire.

Mots utiles

- **Annonce** : court message qu'un appareil diffuse en boucle pour dire « je suis là ». Tout téléphone à portée peut le lire, comme une enseigne lumineuse.
- **RSSI** : force du signal reçu, en dBm. C'est toujours un nombre négatif. Plus il est proche de 0, plus le signal est fort : -40 = tout près, -95 = loin.
- **UUID** : numéro qui indique un type de service. Par exemple 180D = fréquence cardiaque.

Ce que vous faites

1. Ouvrez nRF Connect, onglet **Scanner**.
2. Lancez le scan (**SCAN** ou bouton de lecture).
3. Regardez la liste. Pour chaque appareil : un **nom** (ou « N/A » / « Unnamed »), une **adresse** (Android) ou un identifiant (iOS), et le **RSSI**.
4. Touchez une ligne (Android) ou ouvrez la fiche (iOS). Vous voyez les **données d'annonce** : *Flags, Complete Local Name, Service UUIDs, Manufacturer data, Tx Power Level...*
5. Prenez **un appareil d'un membre de l'équipe** (écouteurs, montre). Notez son RSSI à 30 cm, puis à 3 m, puis derrière une porte. Astuce : selon la version, nRF Connect trace le RSSI dans le temps (vue graphique).
6. Remplissez le tableau ci-dessous pour 5 appareils. Remplacez les noms personnels : « écouteurs d'un membre », « montre », « inconnu ».

#	Nom affiché (anonymisé)	RSSI (dBm)	Services annoncés (UUID)	Données fabricant ?	Ce que l'annonce révèle
1					
...					

Vous devez voir :

- une liste d'appareils, avec des RSSI entre environ -40 dBm (tout près) et -95 dBm (loin, à travers un mur) ;
- beaucoup d'appareils **sans nom** ;
- un RSSI qui baisse (devient plus négatif) quand vous vous éloignez.

Questions (réponses dans analyses/m2_tp3/README.md)

- a. Pourquoi la force du signal (RSSI) ne suffit-elle pas pour mesurer une distance précise ?
- b. Quels noms diffusés pourraient trahir leur propriétaire ? Quel risque si un **dispositif médical** diffusait « Pompe de M. X » ou « Holter chambre 12 » ?
- c. Relancez le scan après quelques minutes. Certaines adresses ont-elles changé ? À quoi cela sert-il ?

Étape 2 — Comprendre GATT sur le service Heart Rate (5 min)

Pourquoi : savoir où se trouve la fréquence cardiaque dans un capteur Bluetooth, pour aller la chercher à l'étape 3.

Ce que vous faites

1. Lisez le schéma ci-dessous, de haut en bas.
2. Repérez la caractéristique qui contient la mesure : **Heart Rate Measurement** (0x2A37).
3. Lisez les 5 définitions sous le schéma.

```
Serveur GATT = le capteur (ceinture, montre, futur ESP32)      Client GATT = le smartphone, la box
└─ Service Heart Rate ..... UUID 0x180D
    └─ Heart Rate Measurement ..... UUID 0x2A37  propriété : NOTIFY
        └─ Descripteur CCCD ..... UUID 0x2902  (le client y écrit pour s'abonner)
            └─ Body Sensor Location (option) ..... UUID 0x2A38  propriété : READ (0x01 torse, 0x02 poignet...)
                └─ Heart Rate Control Point (si énergie) . UUID 0x2A39  propriété : WRITE (remise à zéro de l'énergie)
```

- **GATT** : la façon normalisée dont un appareil Bluetooth range ses données. Image : une boutique, avec des rayons (services) qui contiennent des articles (caractéristiques). Le **serveur** (le capteur) est la boutique : il détient les données. Le **client** (le smartphone) vient les chercher.
- **Service** : un rayon de la boutique. Il regroupe des données qui vont ensemble (ici, tout ce qui concerne le cœur).
- **Caractéristique** : un article du rayon. C'est une donnée, plus ce qu'on a le droit d'en faire : lire, écrire, être prévenu.
- **Notification** : le capteur **envoie** lui-même chaque nouvelle valeur, sans qu'on la demande, comme une alerte SMS. Pour la FC, c'est en général une fois par seconde. Pour s'abonner, le client écrit dans une petite case de réglage : le descripteur **CCCD** (0x2902).
- **UUID sur 16 bits** (0x180D, 0x2A37...) : un numéro officiel, donné par le **Bluetooth SIG** (l'organisme qui gère la norme Bluetooth). Toute application compatible sait donc lire le capteur.

Étape 3 — Se connecter à une source Heart Rate (10 min)

Pourquoi : voir arriver sur un téléphone les vrais octets envoyés par un capteur cardiaque.

Ce que vous faites

1. Choisissez **une** des trois voies, selon ce que l'équipe possède (tableau ci-dessous).
2. Suivez les actions de votre voie.
3. Faites **une capture d'écran** (scan ou service) pour le livrable.

Voie	Il vous faut	Principe
A	Une ceinture cardio ou une montre qui diffuse la FC (un membre volontaire)	Vous vous connectez au vrai capteur
B	Deux téléphones, dont un Android	L'Android joue le capteur, l'autre téléphone le lit
C	iPhone(s) uniquement, ou rien ne fonctionne en 10 min	Vous passez directement à l'étape 4

Voie A — Ceinture cardio ou montre

1. Sur la montre, activez la diffusion de la FC si elle existe (selon les modèles : « diffuser la fréquence cardiaque », « broadcast heart rate » ; à vérifier selon le modèle). Une ceinture cardio Bluetooth diffuse dès qu'elle est portée et humidifiée.
2. Fermez l'application sportive du propriétaire : beaucoup de capteurs n'acceptent qu'une ou deux connexions à la fois.
3. Dans nRF Connect, touchez **CONNECT** sur l'appareil.
4. Ouvrez le service **Heart Rate**, puis la caractéristique **Heart Rate Measurement**.
5. Touchez l'icône d'**abonnement aux notifications** (flèches vers le bas).
6. Relevez **5 trames** en hexadécimal.

Voie B — Deux téléphones, dont un Android : simuler un capteur (menus à vérifier selon la version de l'appli)

1. Téléphone **Android** (le « capteur ») : menu $\equiv \rightarrow$ **Configure GATT server** $\rightarrow$ **Add service** $\rightarrow$ choisissez **Heart Rate** dans les services prédéfinis. La caractéristique Heart Rate Measurement et son CCCD sont ajoutés avec lui. Selon la version, une configuration d'exemple existe déjà : vérifiez qu'elle contient bien 0x180D.
2. Toujours sur Android : onglet **Advertiser** $\rightarrow$ **+** $\rightarrow$ créez une annonce **connectable**.
3. Ajoutez un enregistrement (**Add record**) de type **Service UUID** avec **180D** (et éventuellement le nom), puis activez l'annonce.
4. **Second téléphone** (iOS ou Android, le « smartphone du patient ») : scannez, repérez l'annonce 0x180D, touchez **CONNECT**, abonnez-vous aux notifications de 0x2A37.
5. Retour sur l'Android : un onglet **Server** (ou équivalent) montre le client connecté. Sélectionnez Heart Rate Measurement, tapez une valeur en hexadécimal (par exemple 16-48-2C-03) et

envoyez la notification.

6. Changez la valeur (reprenez les trames de l'étape 4) et regardez ce qu'affiche le second téléphone.

Un téléphone ne peut pas être à la fois le capteur et son propre client : il faut deux appareils.

Voie C — iPhone(s) uniquement, ou rien ne fonctionne en 10 min

1. Passez directement à l'étape 4 (trames fournies) : c'est l'exercice central, faisable par tous.
2. Option sur iPhone (LightBlue, mode périphérique de nRF Connect iOS) — détails : voir l'annexe.

Vous devez voir, sous Heart Rate Measurement, une valeur brute du type `(0x) 16-48-2C-03`. Selon l'appli et le téléphone, vous voyez aussi sa traduction : FC en bpm, état du contact, intervalle RR en ms.

Étape 4 — Décoder 3 trames à la main (10 min, exercice central)

Pourquoi : tout logiciel qui reçoit la mesure fait ce décodage. Le comprendre permet de repérer une valeur absurde.

Mots utiles

- **Trame** : le message envoyé par le capteur. C'est une suite d'**octets**.
- **Octet** : une case de 8 interrupteurs (bits), chacun à 0 ou 1. On l'écrit avec 2 caractères en hexadécimal, par exemple `48`.
- **Hexadécimal** : façon d'écrire les nombres avec 16 chiffres : 0 à 9, puis A à F (A = 10, B = 11, C = 12, D = 13, E = 14, F = 15). Le `0x` devant un nombre veut juste dire « écrit en hexadécimal ».
- **Flags** (drapeaux) : le premier octet. Chacun de ses 8 interrupteurs répond oui (1) ou non (0) à une question. Ensemble, ils annoncent ce que contient la suite du message.
- **Intervalle RR** : le temps entre deux battements du cœur.
- **Little-endian** : « petit bout d'abord ». Sur 2 octets, l'octet des unités arrive en premier.

Ce que vous faites

1. Lisez les 3 outils ci-dessous, puis le format.
2. Suivez l'exemple entièrement décodé `16 48 2C 03`.
3. Complétez le modèle de chaque trame : **3**, puis **5**, puis **2** (de la plus facile à la plus piègeuse).
4. Vérifiez qu'il ne reste aucun octet à la fin.
5. Recopiez vos résultats dans le tableau de réponses (à déposer dans le livrable).
6. Équipe en avance : faites aussi les trames 1, 4 et 6 (bonus). Sinon, le notebook les décode pour vous à l'étape 5.

Trois outils pour lire un octet

Outil 1 — Hexadécimal → nombre. On multiplie le 1er caractère par 16, puis on ajoute le 2e.

48 → $4 \times 16 + 8 = 72$. 2C → $2 \times 16 + 12 = 44$ (car C = 12).

Outil 2 — Hexadécimal → interrupteurs. Chaque caractère donne 4 interrupteurs, qui valent 8, 4, 2 et 1. On allume ceux dont la somme fait le chiffre. 6 = 4 + 2 → 0110. 1 → 0001. Donc 16 → 0001 0110.

Hexa	Valeur	Interrupteurs (8 4 2 1)	Hexa	Valeur	Interrupteurs (8 4 2 1)
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	A	10	1010
3	3	0011	B	11	1011
4	4	0100	C	12	1100
5	5	0101	D	13	1101
6	6	0110	E	14	1110
7	7	0111	F	15	1111

Outil 3 — Little-endian (« petit bout d'abord »). Au-delà de 255, un nombre prend 2 octets. Le capteur envoie d'abord l'octet des unités, puis l'octet des « paquets de 256 ». C'est comme une date écrite jour/mois : le petit bout d'abord. Pour lire, on **retourne** les deux octets. 2C 03 → on lit 03 2C → 3 paquets de 256 + 44 = 768 + 44 = **812**.

Le format (Bluetooth SIG, *GATT Specification Supplement*, § 3.126) : on lit toujours dans cet ordre, en sautant ce qui est absent.

Octet(s)	Ce que c'est	Présent quand ?
1	Flags (drapeaux) : 8 interrupteurs oui / non	toujours
1 ou 2	Fréquence cardiaque (FC), en battements par minute (bpm)	toujours ; 1 octet si interrupteur 0 = 0, 2 octets si interrupteur 0 = 1
2	Énergie dépensée (kJ, cumulée)	interrupteur 3 = 1
2 × n	Intervalles RR, en 1/1024 de seconde (du plus ancien au plus récent)	interrupteur 4 = 1

Les interrupteurs se numérotent de **droite à gauche** : le bit 0 est le plus à droite.

Interrupteur (bit)	Question à laquelle il répond (1 = oui, 0 = non)
0	La FC prend-elle 2 octets ? (0 = 1 octet uint8, 1 = 2 octets uint16)
1	Le capteur touche-t-il bien la peau ? (contact détecté)

Interrupteur (bit)	Question à laquelle il répond (1 = oui, 0 = non)
2	Le capteur sait-il vérifier le contact ? (détection prise en charge ; si 0, ignorez l'interrupteur 1)
3	L'énergie dépensée est-elle dans le message ?
4	Des intervalles RR sont-ils dans le message ?
5-7	Réservés pour plus tard (toujours à 0)

Exemple entièrement décodé, pas à pas : 16 48 2C 03

1. On découpe. 4 octets : 16 | 48 | 2C | 03. Le premier, 16, ce sont les interrupteurs.
2. On allume les interrupteurs du premier octet. 1 → 0001 et 6 = 4 + 2 → 0110. Donc 16 = 0001 0110.

Position	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
Interrupteur	0	0	0	1	0	1	1	0
Ce que ça veut dire	réservé	réservé	réservé	RR présent(s)	pas d'énergie	le capteur sait vérifier le contact	il touche la peau	FC sur 1 octet

En lisant de droite à gauche : - bit 0 = 0 → la FC tient sur **1 octet** ; - bits 1 et 2 = 1 et 1 → **contact détecté** : le capteur sait vérifier le contact, et il touche bien la peau ; - bit 3 = 0 → **pas d'énergie** dans ce message ; - bit 4 = 1 → au moins un **intervalle RR** suit.

3. La FC (1 octet). 48 → $4 \times 16 + 8 = 72$. FC = **72 bpm**.
4. L'énergie. Interrupteur 3 éteint : rien à lire, on passe.
5. Le RR (2 octets, petit bout d'abord). 2C 03 → on retourne : 03 2C → $3 \times 256 + 44 = 812$. L'unité est 1/1024 de seconde : on divise par 1024. $812 / 1024 = 0,793$ s (793 ms) entre deux battements. FC instantanée = $60 / 0,793 \approx 75,7$ bpm.
6. On vérifie. 4 octets lus sur 4 : il ne reste rien. La trame est complète.

Octet(s)	Valeur	Lecture
16	0001 0110	FC sur 1 octet ; contact détecté ; pas d'énergie ; RR présent(s)
48	72	FC = 72 bpm
2C 03	$0x032C = 812$	RR = $812 / 1024 = 0,793$ s (FC instantanée $\approx 75,7$ bpm)

Les 6 trames de l'exercice — vous en décidez **3 à la main** ; le notebook decode les 6.

n°	Trame (hexadécimal)	Ce qui est nouveau	À la main ?
1	00 48	FC sur 1 octet	bonus

n°	Trame (hexadécimal)	Ce qui est nouveau	À la main ?
2	01 5A 00	FC sur 2 octets	oui (3e, la plus piègeuse)
3	06 3E	état du contact	oui (1re, la plus facile)
4	0E 55 E8 03	énergie dépensée	bonus
5	16 4B 20 03	un intervalle RR	oui (2e, comme l'exemple)
6	1E 4E C8 00 F0 02 10 03	énergie + deux intervalles RR	bonus

Modèle à compléter — trame 3 : 06 3E

Octet(s)	Je le transforme	Ce que ça veut dire
06 (interrupteurs)	0 → ; 6 → ; donc 	bit 0 : ; bits 1 et 2 : ; bit 3 : ; bit 4 :
3E	... × 16 + ... =	FC = bpm
Reste-t-il un octet ?	oui / non	

Modèle à compléter — trame 5 : 16 4B 20 03

Octet(s)	Je le transforme	Ce que ça veut dire
16 (interrupteurs)		bit 0 : ; bits 1 et 2 : ; bit 3 : ; bit 4 :
4B	... × 16 + ... =	FC = bpm
20 03	je retourne : → ... × 256 + ... =	RR = / 1024 = s = ms ; FC instantanée = 60 / ≈ bpm
Reste-t-il un octet ?	oui / non	

Modèle à compléter — trame 2 : 01 5A 00

Octet(s)	Je le transforme	Ce que ça veut dire
01 (interrupteurs)		bit 0 : (donc la FC prend ... octet(s)) ; bits 1 et 2 : ; bit 3 : ; bit 4 :
5A 00	je retourne : → ... × 256 + ... =	FC = bpm
Reste-t-il un octet ?	oui / non	

Tableau de réponses (à déposer dans le livrable). Colonne « FC instantanée » : on divise 60 par le RR en secondes. RR en ms = RR en secondes × 1000.

n°	Flags (interrupteurs)	Format FC	Contact	FC (bpm)	Énergie (kJ)	RR (ms)	FC instantanée 60/RR (bpm)
3							
5							
2							

Vous devez obtenir : pour chaque trame, aucun octet restant à la fin, et des valeurs possibles pour un être humain. Une FC ou un RR absurde (par exemple 23 040 bpm ou 8 s entre deux battements) signale une erreur de lecture.

Étape 5 — Vérifier avec Python (5 min)

Pourquoi : contrôler votre décodage à la main avec un programme qui applique la norme.

Dans le notebook, ► veut dire « exécutez sans rien changer » et ✎ « à vous de modifier ». Les cellules marquées « pas besoin de lire ce code » fabriquent juste un outil.

Ce que vous faites

1. Ouvrez `TP_M2_3_BLE_decodeur.ipynb` (Colab ou local) et exécutez tout.
2. Dans la cellule « ✎ À vous », remplacez `None` par **vos** FC trouvées à la main (trames 3, 5 et 2) dans `MES_FC`.
3. Relancez cette cellule.
4. Si vous avez capturé des trames réelles (voies A ou B), collez-les dans `MES_TRAMES_CAPTUREES` (Bonus A du notebook).
5. Répondez aux 5 questions de la section 4 du notebook.

Vous devez voir :

- un tableau de 6 lignes (FC 72, 90, 62, 85, 75, 78 bpm) : le notebook décode aussi les trames que vous n'avez pas faites ;
- la mention `OK` pour chaque FC que vous avez décodée à la main ;
- le message `Tous les tests sont passés.`

Étape 6 — Et au projet 1 ? (lecture, 2 min)

Pourquoi : savoir ce que votre propre objet devra envoyer le 30 octobre.

Ce que vous faites

1. Lisez le paragraphe ci-dessous.
2. Parcourez le squelette de programme en annexe (§ 8). Repérez la ligne qui fabrique l'octet d'interrupteurs (`0x16`).

Le 30 octobre, c'est **votre ESP32** qui jouera le rôle du capteur. Il proposera le service `0x180D`. Il fabriquera lui-même ces octets, à partir de la FC calculée sur le signal du MAX30102 (filtres du

TP 1). Wokwi ne simule pas le Bluetooth : ce code se teste sur la vraie carte, avec nRF Connect comme client.

6. Livrable (fin de séance, dans le repo de l'équipe)

Dossier `analyses/m2_tp3/` :

1. `trames_decodees.csv` (les 6 trames, exporté par la cellule « Livrable » du notebook) **et** votre tableau des 3 trames rempli à la main (`trames_decodees.md`, ou photo lisible de la feuille) ;
2. `capture_nrfconnect.png` : une capture du scan **ou** du service Heart Rate (noms personnels floutés) ;
3. `README.md` : voie utilisée (A, B ou C), le tableau des 5 appareils scannés (anonymisé), les réponses aux questions a-c de l'étape 1 et aux 5 questions du notebook (2 lignes chacune).

Ce livrable compte dans la part TP de la note du module 2.

7. Pour aller plus loin

- **Variabilité de la FC** : récupérez une vingtaine d'intervalles RR (voie A). Calculez le RMSSD : on prend les écarts entre deux RR qui se suivent, on les met au carré, on fait la moyenne, puis la racine. Pourquoi un seul battement manqué fausse-t-il fortement le résultat ?
- **Autres services santé normalisés** : Health Thermometer (0x1809, caractéristique 0x2A1C, valeur au format FLOAT IEEE 11073), Pulse Oximeter (0x1822), Battery (0x180F, niveau en % dans 0x2A19). Cherchez leur format dans la *GATT Specification Supplement*.
- **Sécurité** : la plupart des ceintures cardio n'exigent ni appairage ni chiffrement. N'importe quel téléphone à portée peut donc s'abonner. Acceptable pour le sport, pas pour un dispositif médical. On y reviendra au projet 2 (appairage, chiffrement du lien, modèle de menaces).
- **Programmation** : comparez la bibliothèque Bluetooth intégrée au cœur Arduino ESP32 et **NimBLE-Arduino** (plus légère en mémoire).

8. Annexe — Squelette d'un serveur Heart Rate sur ESP32 (projet 1)

Lecture facultative aujourd'hui : à l'étape 6, repérez seulement la ligne `trame[0] = 0x16;`. Ce code servira au projet 1.

FC **simulée** (aucun capteur) : au projet 1, remplacez-la par la FC calculée depuis le MAX30102 et positionnez le bit « contact » selon la détection du doigt. Bibliothèque BLE intégrée au cœur Arduino ESP32 (`BLEDevice`) : aucune bibliothèque à installer. **Ne fonctionne pas dans Wokwi** (pas de simulation Bluetooth) : à flasher sur la carte, puis à tester avec nRF Connect (scan → Digi5-HR-eq01 → CONNECT → notifications de 0x2A37).

```
// Serveur BLE « Heart Rate » (service 0x180D) sur ESP32 – squelette du projet 1
// FC SIMULÉE (aucun capteur) : au P1, remplacez-la par la FC calculée depuis le MAX301
```

```

02.
// Données fictives, usage pédagogique, non diagnostique.
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define NOM_BLE "Digi5-HR-eq01" // nom diffusé : jamais le nom d'une personne (RGPD)

static BLEUUID UUID_SERVICE_HR((uint16_t)0x180D); // Heart Rate (service)
static BLEUUID UUID_HRM((uint16_t)0x2A37);        // Heart Rate Measurement
static BLEUUID UUID_BSL((uint16_t)0x2A38);        // Body Sensor Location

BLECharacteristic *hrm = nullptr;
bool connecte = false;
uint32_t dernierEnvoi = 0;

class ServeurCallbacks : public BLEServerCallbacks {
    void onConnect(BLEServer *serveur) override {
        connecte = true;
        Serial.println("Central connecte");
    }
    void onDisconnect(BLEServer *serveur) override {
        connecte = false;
        Serial.println("Central deconnecte : reprise des annonces");
        BLEDevice::startAdvertising();
    }
};

void setup() {
    Serial.begin(115200);
    BLEDevice::init(NOM_BLE);

    BLEServer *serveur = BLEDevice::createServer();
    serveur->setCallbacks(new ServeurCallbacks());
    BLEService *service = serveur->createService(UUID_SERVICE_HR);

    // Heart Rate Measurement : notification uniquement (pas de lecture directe)
    hrm = service->createCharacteristic(UUID_HRM, BLECharacteristic::PROPERTY_NOTIFY);
    hrm->addDescriptor(new BLE2902()); // CCCD 0x2902 : le client y écrit pour activer les notifications

    // Body Sensor Location : lecture, 0x02 = poignet (Wrist)
    BLECharacteristic *bsl = service->createCharacteristic(UUID_BSL, BLECharacteristic::PROPERTY_READ);
    uint8_t poignet = 0x02;

```

```

    bsl->setValue(&poignet, 1);

    service->start();
    BLEAdvertising *annonces = BLEDevice::getAdvertising();
    annonces->addServiceUUID(UUID_SERVICE_HR); // le 0x180D apparaît dans l'annonce
    annonces->setScanResponse(true);           // le nom part dans la réponse au scan
    BLEDevice::startAdvertising();
    Serial.println("Annonces BLE lancees");
}

void loop() {
    if (connecte && millis() - dernierEnvoi >= 1000) { // une notification par seconde
        dernierEnvoi = millis();
        uint8_t fc = 70 + random(0, 10);                // FC simulée (bpm)
        uint16_t rr = (uint16_t)(60.0 / fc * 1024.0 + 0.5); // RR en 1/1024 s, déduit de
        la FC (simulation)
        uint8_t trame[4];
        trame[0] = 0x16; // flags 0001 0110 : FC uint8, contact pris en charge et dé
        tecté, RR présent
        trame[1] = fc; // FC sur 1 octet
        trame[2] = rr & 0xFF; // RR, octet de poids faible d'abord (little-endian)
        trame[3] = rr >> 8; // RR, octet de poids fort
        hrm->setValue(trame, sizeof(trame));
        hrm->notify();
        Serial.printf("Notifie : %02X %02X %02X %02X\n", trame[0], trame[1], trame[2], tram
        e[3]);
    }
}

```

À noter : - **Le nom diffusé** ne doit identifier ni le patient ni l'établissement : préférez un identifiant d'appareil. - La ligne `addDescriptor(new BLE2902())` sert avec la pile Bluetooth « Bluedroid » (pile par défaut de l'ESP32 classique). Avec la pile **NimBLE**, le CCCD est ajouté automatiquement et `BLE2902` est marqué obsolète : supprimez la ligne si le compilateur le signale. - **Énergie** : la radio BLE active coûte des dizaines de mA (voir TP 4) ; au projet 1, on mesurera ce firmware au wattmètre USB.

Annexe — Détails techniques

Vous pouvez sauter cette annexe : elle sert seulement si vous travaillez sans Colab, ou sur iPhone en voie C.

Python en local (au lieu de Colab) : `Python 3.10+`, `pip install numpy scipy pandas matplotlib jupyter`, puis `jupyter notebook` dans le dossier du fichier.

Où trouver le notebook : le dépôt GitHub du cours est privé : prenez le notebook sur Teams / Moodle.

Voie C, options sur iPhone : l'application **LightBlue** (iOS) permet de créer un périphérique virtuel (« Virtual Devices ») ; un modèle Heart Rate y est proposé selon la version (à vérifier). Les versions récentes de nRF Connect iOS annoncent aussi un mode périphérique (à vérifier).